Projeto Final: Labirinto com Chaves (Grafo de Estados) + Comparação de Algoritmos

Teoria dos Grafos + Análise e Visualização de Dados (AVD) — 2026.2

Python 3.11+ (obrigatório; outras linguagens não serão aceitas, porque a parte visual precisa estar integrada ao código). Proibido usar libs que já implementem os algoritmos (ex.: `networkx`, `igraph`, `graph-tool`) para **BFS/DFS/Dijkstra/Bellman-Ford**. Pode usar `pandas` (IO), `argparse`, `heapq`, `dataclasses`, `typing`, `matplotlib/plotly/pyvis/streamlit` **apenas** para visualização/UX. **Exceção:** `networkx` pode ser usado **somente dentro de** `tests/`, como oráculo para conferir os resultados da implementação de vocês.

Entrega (GitHub obrigatório)

- Subam todo o projeto em um **repositório GitHub** e entreguem **apenas o link** no Classroom.
- Incluam: código, mapas, dataset (ou script de download), **README** com instruções, **PDF** (manual + técnica) e a **Declaração de Uso de IA** assinada (`docs/DECLARACAO_IA.pdf`).
- **Grupos de 4 ou 5 integrantes**, com divisão de tarefas por pacotes de trabalho (cada pacote tem um dono e um revisor). Todos os integrantes devem ter commits em pelo menos 2 pacotes, e o PDF deve trazer a **matriz de contribuição**.

Calendário

Etapa	Data
Proposta do dataset da Parte 2 (checkpoint)	até 12/10/2026, via Classroom (avaliada pelas duas professoras)
Acompanhamentos em sala (Grafos)	10/11, 17/11, 19/11 e 24/11
Acompanhamentos em sala (AVD)	17/11, 19/11 e 24/11
Entrega do material (todos os integrantes postam)	26/11/2026, via Classroom
Apresentações	26/11 (qui): turma A · 01/12 (ter): turma B · 03/12 (qui): remanescentes e quem cursa só AVD

A ordem das apresentações é sorteada em sala. Grupos com integrantes que têm aula de outra disciplina no horário apresentam por último. Cada grupo tem de **15 a 20 minutos**.

Estrutura de pastas (obrigatória)

```
projeto-grafos/
├─ README.md
├─ requirements.txt  (ou pyproject.toml)
├─ docs/
│  └─ DECLARACAO_IA.pdf   # declaração de uso de IA assinada por todos
├─ data/
│  ├─ mapas/              # mapas fornecidos (m01 a m08)
│  ├─ mapas_grupo/        # 2 mapas criados pelo grupo (Parte 1, Seção 10)
│  └─ dataset_parte2/     # dataset maior (Parte 2)
```

```
├ out/
│   ├ parte1/
│   └ parte2/
├ src/
│   ├ cli.py
│   ├ solve.py
│   ├ graphs/
│   │   ├ io.py            # ler/validar os mapas e o dataset
│   │   ├ estados.py       # grafo de posições e grafo de estados
│   │   ├ graph.py         # lista de adjacência DIRIGIDA
│   │   └ algorithms.py    # BFS, DFS, Dijkstra, BF (próprios)
│   └ viz.py               # visualizações/UX
└ tests/                   # (obrigatórios, mínimos)
    ├ test_bfs.py
    ├ test_dfs.py
    ├ test_dijkstra.py
    ├ test_bellman_ford.py
    ├ test_estados.py
    └ test_io.py
```

PARTE 1

Labirinto com Chaves e Portas (Grafo de Estados)

Ideia central

O mapa do labirinto **não é o grafo**. Uma mesma casa do mapa corresponde a **vários nós**, um para cada situação em que o jogador pode chegar nela (quais chaves já tem, quais bônus já consumiu). Construir esse grafo corretamente é o coração da Parte 1.

O que há nos mapas

Os arquivos `data/mapas/*.txt` são grades retangulares de caracteres. Linhas iniciadas com `;` são comentários. Símbolos:

Símbolo	Significado	Custo de entrar
`#`	Parede (intransponível)	—
`.`	Chão	1
`S`	Início (exatamente um por mapa)	1
`G`	Saída (pelo menos uma). Ao entrar em G o jogo termina	1
`,`	Lama	3
`~`	Água	5
`a` a `d`	Chave. Ao entrar, o jogador pega a chave (fica com ela para sempre)	1
`A` a `D`	Porta. Só pode entrar quem tem a chave minúscula correspondente. A porta não fecha e a chave não é gasta	1
`> < ^ v`	Esteira. Pode-se entrar por qualquer lado; estando nela, o único movimento permitido é na direção da seta	1
`+`	Bônus consumível: vale −4 na primeira vez que o jogador entra; depois se comporta como chão	−4, depois 1
`*`	Fonte de energia: vale −2 toda vez que o jogador entra (não se esgota)	−2

Regras de movimento (obrigatórias: o gabarito depende delas)

- Movimentos em 4 direções, sempre testadas na ordem **N, L, S, O** (cima, direita, baixo, esquerda). Essa ordem define a ordem dos vizinhos na lista de adjacência.
- **Custo de um movimento = custo da casa de destino** (tabela acima).
- Não é possível sair da grade nem entrar em parede.
- Estados posicionados em `G` **não têm arestas de saída** (o jogo terminou).

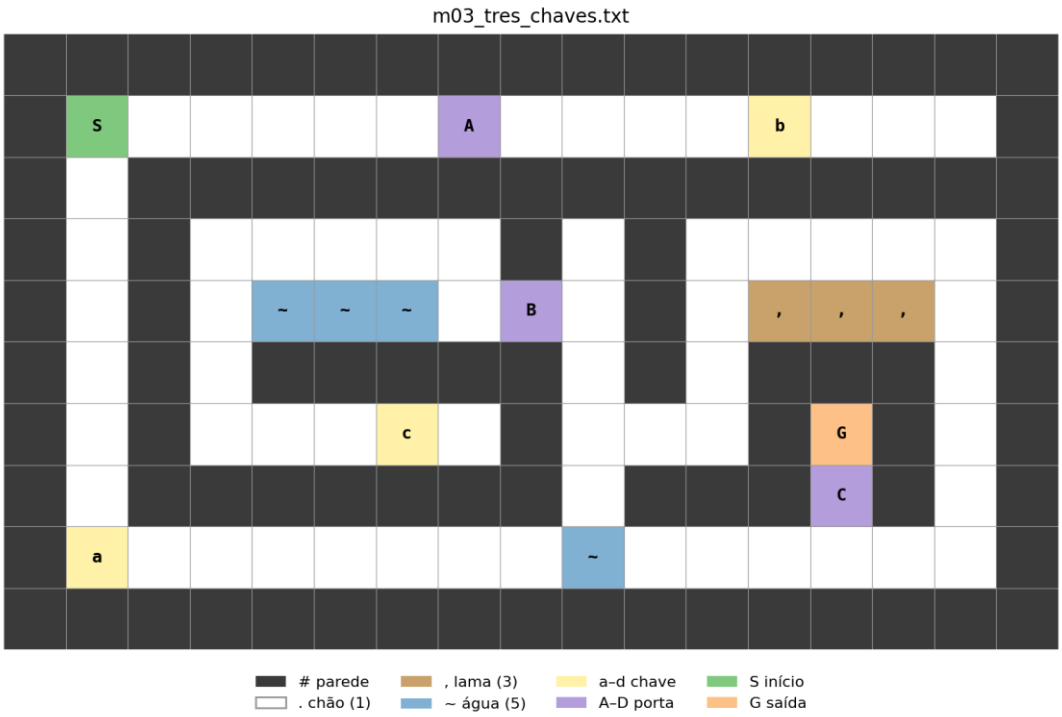


Figura 1: o mapa m03_tres_chaves.txt. As portas obrigam uma ordem de coleta: a → b → c.

Mapas fornecidos

Arquivo	O que ele exercita
`m01_terreno.txt`	Custos de terreno: o caminho com menos passos não é o mais barato
`m02_uma_chave.txt`	Uma chave e uma porta (exemplo da Figura 2)
`m03_tres_chaves.txt`	Três chaves com ordem de coleta obrigatória
`m04_esteiras.txt`	Arcos de mão única, becos e estados-armadilha
`m05_bonus.txt`	Bônus consumíveis: pesos negativos sem ciclo negativo
`m06_fonte.txt`	Fonte de energia: ciclo negativo (um "exploit" do jogo)
`m07_impossivel.txt`	Mapa sem solução
`m08_grande.txt`	Mapa 31×71 com 4 chaves

1) NÓS: dois grafos para o mesmo mapa

Para cada mapa, construam **dois** grafos:

(a) **Grafo de posições** (`G_pos`): cada nó é uma casa livre `(linha, coluna)`; os arcos são os movimentos permitidos pela geometria (paredes e esteiras). É o modelo "ingênuo", usado para comparação na Seção 6.

(b) **Grafo de estados** (`G_est`): cada nó é um estado **(linha, coluna, chaves, bônus)**, em que "chaves" é o conjunto de chaves que o jogador possui e "bônus" é o conjunto de bônus + já consumidos (numerados de 0 em diante, em ordem de leitura: linha, depois coluna).

Rótulo obrigatório dos estados: `(3,5)|ab|{0,2}`. Sem chaves, use `-`; sem bônus, use `{}`. Exemplo: o estado inicial de m02 é `(1,1)|-|{}`.

Regras de construção do grafo de estados:

- O grafo deve ser construído **por exploração** (BFS ou DFS) a partir do estado inicial: só entram estados **alcançáveis**. Não gerem todas as combinações de antemão.
- Reportem o **teto teórico** (casas livres × 2^(nº de chaves) × 2^(nº de bônus)) e comparem com o número de estados realmente alcançáveis.
- Criem um **nó virtual** `ALVO`, com arco de custo 0 saindo de todo estado posicionado em `G`. Os caminhos são sempre calculados até `ALVO`. As métricas da Seção 3 **não** contam esse nó.



Figura 2: em m02 o jogador passa duas vezes pelas mesmas casas, mas em estados diferentes. Sem a chave, A funciona como parede; com a chave, o mesmo caminho vira outro nó do grafo.

2) ARESTAS: geradas pelo código, não escritas à mão

As arestas não são escritas à mão: elas são derivadas das regras do jogo pelo código de vocês. Para cada mapa, exportem `out/parte1/arestas_<mapa>.csv`:

```
origem,destino,movimento,casa_destino,peso
"(1,1)|-|{}","(2,1)|-|{}",S,.,1
"(3,10)|-|{}","(3,11)|a|{}",L,a,1
"(1,5)|a|{}","(1,6)|a|{}",L,A,1
...
```

- Grafo **dirigido**: uma linha por arco, **sem espelhamento**. Expliquem no PDF por que um grafo não dirigido estaria errado aqui (pensem em esteiras e em coletar uma chave).
- O arquivo do m08 também deve ser gerado.

3) Métricas globais e por camada

Definições (grafo dirigido):

- **Ordem** = |V|; **Tamanho** = |E| (número de arcos).
- **Densidade** `(dirigido) = |E| / (|V| (|V| − 1))`. `Se |V| < 2, densidade = 0.`

- **Grau de entrada** e **grau de saída** de cada nó.

Peçam e entreguem:

1. **Por mapa:** ordem, tamanho e densidade de `G_pos` e de `G_est`; teto teórico; taxa de ocupação = |V(G_est)| / teto.
2. **Camadas (subgrafos induzidos):** para cada conjunto de chaves que aparece em `G_est`, considerem o subgrafo induzido pelos estados com aquele conjunto e calculem ordem, tamanho e densidade.
3. **Arcos entre camadas:** quantos arcos ligam camadas diferentes e quais casas do mapa os geram. Expliquem por que são exatamente essas casas.

Arquivos de saída (obrigatórios):

- `out/parte1/metricas_global.json` (todos os mapas)
- `out/parte1/camadas.json` (métricas por camada, por mapa)
- `out/parte1/graus_<mapa>.csv` → `estado, grau_entrada, grau_saida`

4) Graus e rankings

- Estado com **maior grau de saída** e com **maior grau de entrada** em cada mapa (desempate pela ordem de leitura do rótulo).
- **Multiplicidade de uma casa:** número de estados de `G_est` que correspondem àquela casa. Qual é a multiplicidade máxima no m08, quantas casas a atingem e por quê? (Será usada no heatmap da Seção 8.)
- **Becos:** estados com grau de saída 0 que **não** estão em G (por exemplo, uma esteira apontando para a parede).

5) Pesos: fixos pelas regras

Os pesos não são escolhidos pelo grupo: são os da tabela de símbolos, para que todos os grupos possam ser conferidos com o mesmo gabarito. A criatividade fica nos mapas do grupo (Seção 10).

- Dijkstra deve **recusar** (com mensagem clara) qualquer grafo com arco de peso negativo, como m05 e m06.

6) Por que o grafo de posições não basta

Para **m02, m03 e m04**, calculem o menor número de passos (BFS) de S até G em três modelagens:

(a) `G_pos` com **portas tratadas como chão**;

(b) `G_pos` com **portas tratadas como parede**;

(c) `G_est` (modelagem correta).

Entreguem `out/parte1/comparacao_modelagem.csv` → `mapa, abordagem, passos, caminho_valido, observacao` e expliquem no PDF por que (a) produz caminho **inválido** e (b) **falha**.

Pesos negativos: no m05, rodem Bellman-Ford em `G_pos` tratando `+` como `−4` fixo e mostrem que aparece um ciclo negativo. Depois rodem em `G_est` e mostrem que o ciclo some. Escrevam no PDF um **argumento curto (prova)** de por que

`G_est` do m05 não pode ter ciclo negativo. Dica: o que acontece com o conjunto de bônus ao longo de um caminho?

7) Caminhos no grafo de estados

Para cada mapa (m01 a m08 e os mapas do grupo), rodem no `G_est`:

- **BFS:** menor número de passos até `ALVO`.
- **Dijkstra:** menor custo até `ALVO` (ou registro da recusa quando houver peso negativo).
- **Bellman-Ford:** menor custo no m05; no m06, **detecção e exibição do ciclo negativo** (sequência de estados e custo do ciclo).
- **DFS** a partir do estado inicial: classificação dos arcos em **árvore, retorno, avanço e cruzamento** (usando a ordem N, L, S, O) e indicação de existência de ciclos.
- **Estados-armadilha:** estados alcançáveis a partir do início dos quais `ALVO` é **inalcançável**. Obrigatório para m04 e m07. Dica: façam a busca a partir de `ALVO` no **grafo transposto**.

Discussões obrigatórias no PDF:

- **m01:** por que BFS e Dijkstra dão caminhos diferentes?
- **m06:** o ciclo negativo é um "exploit" do jogo. Proponham uma **correção de regra** (por exemplo, tornar `*` consumível), implementem-na como opção `--fonte-consumivel` e mostrem que Bellman-Ford passa a dar uma resposta finita.
- **m07:** demonstrem, pela busca, que não há solução, e relatem quantos estados são alcançáveis.

Arquivos de saída (obrigatórios):

- `out/parte1/solucoes.csv` → `mapa, algoritmo, passos, custo, status, caminho` (status: `OK`, `RECUSA`, `CICLO_NEGATIVO`, `SEM_SOLUCAO`)
- `out/parte1/percurso_m03.json` (caminho de menor custo, estado a estado)
- `out/parte1/ciclo_negativo_m06.json`
- `out/parte1/armadilhas_m04.csv` e `out/parte1/armadilhas_m07.csv`
- `out/parte1/dfs_classificacao.csv` → `mapa, arvore, retorno, avanco, cruzamento`

8) Visualizações da Parte 1

- **Percurso sobre o mapa** (m03 e m08): o caminho de menor custo desenhado sobre a grade, com a **cor do caminho mudando a cada chave coletada**. Arquivos `out/parte1/percurso_m03.png` e `out/parte1/percurso_m08.png`.
- **Grafo de estados do m02 desenhado em camadas** (uma camada por conjunto de chaves), destacando os arcos que mudam de camada.
- **Árvore BFS** de estados do m02, com os níveis visíveis.
- **Heatmap de multiplicidade** das casas do m08 (Seção 4).

Requisito: título, legenda e rótulos em todas as figuras.

9) Apresentação interativa

Entreguem `out/parte1/labirinto_interativo.html` (ou um app `streamlit`) com:

- seleção de mapa e de algoritmo;
- **animação passo a passo** do percurso, mostrando o estado atual (posição, chaves, bônus consumidos, custo acumulado);
- realce dos estados-armadilha e alternância entre camadas;
- velocidade que permita acompanhar o percurso (controle de velocidade ou avanço passo a passo pelo usuário). Animação rápida demais para ser acompanhada não atende ao requisito.

10) Mapas do grupo

Criem **dois mapas próprios** em `data/mapas_grupo/`:

1. Um mapa com **pelo menos 3 chaves, esteiras e bônus**, em que o caminho de menor custo **coleta ao menos um bônus e passa por ao menos uma esteira**.
2. Um mapa **armadilha**, em que **pelo menos 20%** dos estados alcançáveis sejam estados-armadilha.

Para cada mapa, documentem no PDF o gabarito do grupo: número de estados, número de arcos, passos (BFS) e custo (Dijkstra ou Bellman-Ford). **Mapas iguais ou muito semelhantes entre grupos serão tratados como plágio.**

PARTE 2

Dataset Maior, Comparação de Algoritmos e Análise Visual

Escolham **livremente** um dataset maior de grafos (redes sociais, transporte terrestre, coautoria, dependências, comércio, esportes, finanças etc.). **Restrição importante: não é permitido utilizar datasets de malha aérea (voos, aeroportos, rotas aéreas) nem labirintos/grades gerados.** Construam o grafo **sem** libs de algoritmos prontos e comparem **BFS, DFS, Dijkstra e Bellman-Ford** em **corretude e desempenho**.

Requisitos do dataset

- Grafo **dirigido e ponderado**. Se a fonte for não dirigida, o grupo deve justificar como derivou a direção.
- **Pelo menos 20.000 arestas.** Acima de ~500 mil, justifiquem e mostrem a estratégia de amostragem.
- **Pesos negativos que venham dos dados**, com significado no domínio. **Não vale sortear ou inventar valores negativos.** O grupo deve explicar o que significam, no domínio escolhido, um "caminho mínimo" e um "ciclo negativo".
- **Pergunta de pesquisa:** cada grupo formula uma pergunta sobre o domínio que os algoritmos ajudam a responder.
- **Checkpoint de aprovação:** até **12/10/2026**, via Classroom, o grupo entrega uma proposta de uma página com fonte, |V|, |E|, tipo (dirigido/ponderado), origem dos pesos negativos, principais atributos e pergunta de pesquisa. A proposta é avaliada pelas **duas professoras** (Grafos e AVD) e o dataset só pode ser usado após aprovação. **Dois grupos não podem usar o mesmo dataset.**

Sugestões de datasets com pesos negativos (ponto de partida; o grupo pode propor outro):

- **Redes de confiança assinadas (SNAP):** `Bitcoin OTC` e `Bitcoin Alpha` (`usuários avaliam outros de −10 a +10`), `Epinions` (`confiança/desconfiança; grande, exige amostragem`). `snap.stanford.edu/data/soc-sign-bitcoin-otc.html`, `…/soc-sign-bitcoin-alpha.html`, `…/soc-sign-epinions.html`
- **Reddit Hyperlink Network (SNAP):** links entre comunidades com sentimento positivo ou negativo. snap.stanford.edu/data/soc-RedditHyperlinks.html
- **Câmbio:** `peso = −log(taxa)`; `ciclo negativo = oportunidade de arbitragem.`
- **Esportes:** saldo de gols ou de pontos em confrontos diretos.
- **Mercado:** variação de preço ou de estoque entre períodos.

Obrigatório

1. **Descrever e explorar o dataset** sob dois pontos de vista:
    - **grafos:** |V|, |E|, tipo, distribuição de graus de entrada e de saída, origem e significado dos pesos;

    - **dados (AVD):** exploração pelos quatro pilares vistos em AVD: estrutural, distribucional, relacional e detecção de outliers. **Dados faltantes não podem simplesmente ser apagados:** discutam o mecanismo de ausência e justifiquem o tratamento. **Os outliers também precisam ser avaliados**, discutidos e tratados ou excluídos de acordo.
2. **Implementar e rodar:**
    - **BFS/DFS** a partir de ≥ 3 fontes distintas (relatar ordem, camadas e ciclos).
    - **Dijkstra** com pesos ≥ 0 (≥ 5 pares origem–destino). Expliquem qual transformação dos pesos foi usada para essa versão.
    - **Bellman-Ford** com pelo menos **um caso** com peso negativo e **sem** ciclo negativo, e **um** com **ciclo negativo** (detectado e exibido). Se o dataset completo não tiver ciclo negativo, construam o caso a partir de um recorte ou transformação **documentada** dos dados.
3. **Métricas de desempenho:** tempo (média de pelo menos 5 execuções) e, opcionalmente, memória, por algoritmo e tarefa (tabela `out/parte2/parte2_report.json`).
4. **Escalabilidade:** rodem os quatro algoritmos em pelo menos **três tamanhos** do grafo (ex.: 10%, 50% e 100% dos nós) e comparem o crescimento do tempo com a complexidade teórica de cada um.
5. **Discussão crítica:** quando e por que cada algoritmo é mais adequado; limites do seu design de pesos; resposta à pergunta de pesquisa.
6. **Análise visual dos resultados:**
    - visualizações que comparem os algoritmos (tempo, comportamento, caminhos);
    - gráficos adequados para comparação (ex.: barras, linhas);
    - clareza e legibilidade.

Análise exploratória e explanatória dos dados (AVD)

Com base no grafo da Parte 2, realizem uma análise exploratória e explanatória dos dados. A análise deve incluir:

- exploração das métricas do grafo (graus, densidade, centralidade implícita);
- comparações entre grupos ou categorias do dataset;
- identificação de padrões (ex.: hubs, concentração de conexões, distribuição dos pesos negativos);
- **exploração multivariada quando aplicável:** mapeiem a relação do atributo-alvo (ou da variável central da pergunta) com os demais atributos, não apenas com um ou dois, e interpretem essas relações (direta ou não, linear ou não, forte ou moderada ou fraca). Se o dado for univariado, façam a exploração univariada completa.

Produzam **no mínimo 6 visualizações**, sendo:

- **4 visualizações exploratórias** (entendimento do comportamento dos dados);
- **2 visualizações explanatórias** (com foco em comunicação de insights), sendo que **pelo menos uma responde à pergunta de pesquisa**.

As visualizações devem seguir os princípios da disciplina de Análise e Visualização de Dados (AVD):

- organização visual clara (hierarquia, legibilidade, uso adequado de cores);
- escolha adequada do tipo de gráfico para o dado apresentado;
- evitar distorções visuais ou gráficos inadequados.

Requisitos obrigatórios:

- cada visualização deve conter título, legenda e identificação dos eixos;
- deve haver coerência entre o dado e o tipo de gráfico escolhido;
- não utilizar gráficos sem justificativa;
- **gráficos de pizza e de radar só com justificativa muito forte**: na grande maioria dos cenários eles não são a melhor escolha.

Da exploratória à explanatória: a análise explanatória não é um gráfico isolado com título bonito. Ela deve nascer de algo encontrado na exploratória. Registrem no PDF o raciocínio: o que foi encontrado na exploração, qual história querem contar a partir disso (storytelling), por que essa mensagem é a mais relevante e qual técnica usaram para destacá-la.

As visualizações explanatórias devem:

- ser interpretáveis por alguém que não conhece o projeto;
- justificar a escolha do tipo de gráfico, a seleção dos elementos exibidos e os atributos pré-atentivos usados para destacar a mensagem (cor, tamanho, posição etc.);
- destacar claramente a mensagem principal;
- utilizar boas práticas de design analítico.

Entreguem as imagens/HTML em `out/parte2/` + **uma nota analítica curta para cada visualização**, respondendo: o que está sendo mostrado; qual insight pode ser extraído; por que aquele tipo de visualização foi escolhido. A análise deve ser descrita no PDF com interpretação dos resultados. **A interpretação dos gráficos deve ser autoral: usar IA para analisar ou explicar gráficos é proibido.**

Como executar

Exemplos (vocês implementam a orquestração em `src/cli.py`/`src/solve.py`):

```
# PARTE 1 (labirintos)
python -m src.cli --mapa ./data/mapas/m03_tres_chaves.txt --alg BFS --out ./out/parte1/
python -m src.cli --mapa ./data/mapas/m03_tres_chaves.txt --alg DIJKSTRA --out
./out/parte1/
python -m src.cli --mapa ./data/mapas/m05_bonus.txt --alg BELLMAN_FORD --out
./out/parte1/
python -m src.cli --mapa ./data/mapas/m06_fonte.txt --alg BELLMAN_FORD --fonte-
consumivel --out ./out/parte1/
python -m src.cli --mapa ./data/mapas/m04_esteiras.txt --armadilhas --out ./out/parte1/
python -m src.cli --mapa ./data/mapas/m03_tres_chaves.txt --interactive --out
./out/parte1/


# PARTE 2 (dataset maior)
python -m src.cli --dataset ./data/dataset_parte2/ --alg DIJKSTRA --source A --target Z
--out ./out/parte2/
python -m src.cli --dataset ./data/dataset_parte2/ --benchmark --out ./out/parte2/
```

Saídas obrigatórias (resumo)

- `out/parte1/arestas_<mapa>.csv`, `metricas_global.json`, `camadas.json`, `graus_<mapa>.csv`
- `out/parte1/comparacao_modelagem.csv`, `solucoes.csv`, `dfs_classificacao.csv`
- `out/parte1/percurso_m03.json`, `ciclo_negativo_m06.json`, `armadilhas_m04.csv`, `armadilhas_m07.csv`
- `out/parte1/percurso_m03.png`, `percurso_m08.png` + demais figuras da Seção 8
- `out/parte1/labirinto_interativo.html` (ou app streamlit)
- `out/parte2/parte2_report.json` + visualizações da Parte 2 (+ notas analíticas no PDF)

Testes mínimos (pytest)

- **BFS:** níveis corretos em grafo pequeno.
- **DFS:** detecção de ciclo e classificação de arcos em grafo **dirigido** (árvore, retorno, avanço, cruzamento).
- **Dijkstra:** caminhos corretos com pesos ≥ 0; **recusar** dado com peso negativo.
- **Bellman-Ford:** (i) com pesos negativos **sem** ciclo negativo → distâncias corretas; (ii) com **ciclo negativo** → flag **e o ciclo retornado**.
- **Estados:** um mapa mínimo criado pelo grupo (até 5×7), com número de estados e de arcos calculado **à mão** no PDF e conferido pelo teste; esteira apontando para parede gera estado sem saída; coletar uma chave muda de camada.
- **IO:** mapas inválidos (não retangular, dois `S`, nenhum `G`, símbolo desconhecido) geram erro com mensagem clara.

O que vai ser avaliado

Teoria dos Grafos (10,0 pts + até 1,0 de bônus = máximo 11,0)

1. **Parte 1: Grafo de estados** (qualidade técnica e completude): **3,0 PONTOS**
   - Construção de `G_pos` e `G_est`, arestas geradas, métricas, camadas e graus.
   - Comparação de modelagens (Seção 6) e prova do m05.
   - Caminhos com os 4 algoritmos, estados-armadilha, correção do m06.
   - Mapas do grupo, visualizações e HTML interativo.
2. **Parte 2: Dataset maior, comparação e análise visual: 3,0 PONTOS**
   - Dataset aprovado, pesos negativos justificados pelos dados.
   - Execução correta dos 4 algoritmos, casos com peso negativo e ciclo negativo (BF).
   - Métricas de desempenho, escalabilidade e discussão crítica.
3. **Apresentação:** participação nas reuniões de acompanhamento, apresentação e comprometimento com o projeto. Na apresentação, o grupo roda o próprio código **ao vivo** em um **mapa surpresa** fornecido pela professora: **2,0 PONTOS**
4. **Qualidade do código, organização, testes, README, PDF e declaração de uso de IA: 2,0 PONTOS**

Grupo que entregar apenas uma das partes (só Parte 1 ou só Parte 2) terá a nota de Apresentação zerada.

Os monitores acompanham 1 ou 2 grupos cada e verificam repositório, execução, testes, commits e contribuição de todos os integrantes. A rubrica detalhada (critérios, pesos e níveis) é publicada junto com este enunciado.

Bônus Visual/UX: até **+1,0** (a nota do projeto pode chegar a **11,0**) por experiência interativa caprichada (animação do percurso, troca de camadas, editor de mapas, filtros etc.).

Análise e Visualização de Dados (AVD)

Os elementos da Parte 2 ligados a dados e visualização são avaliados pela docente de AVD. A nota do projeto pode chegar a **11,0** ao considerar o bônus visual/UX (também aplicáveis à avaliação de grafos). Relembrando que para AVD o projeto constitui 60% da nota da segunda avaliação, sendo os outros 40% referentes a atividades práticas.

Pontuação máxima (10,0 pts):

Critério	Pontos	O que será observado
Exploração dos dados	3,0	Estrutura e tipos dos atributos, qualidade (dados faltantes, inconsistências, outliers), mecanismo de ausência e tratamento justificado, distribuições e relações multivariadas com o atributo-alvo.
Da exploratória à explanatória (storytelling)	2,0	A explanatória nasce de algo encontrado na exploratória; raciocínio registrado; história clara e relevante para a pergunta de pesquisa.

Critério	Pontos	O que será observado
Boas práticas visuais	2,0	Tipo de gráfico adequado ao dado, seleção de elementos e atributos pré-atentivos justificados, título, legenda e eixos, sem distorções; pizza e radar só com justificativa forte.
Notas analíticas e relatório	1,5	Nota analítica autoral para cada visualização (o que mostra, insight, por que o gráfico) e interpretação no PDF; resposta à pergunta de pesquisa.
Apresentação e defesa das visualizações	1,5	Capacidade de explicar e justificar cada visualização e cada decisão de análise na arguição.
Total	10,0	

Bônus Visual/UX: até **+1,0** (a nota do projeto pode chegar a **11,0**) por experiência interativa caprichada (animação do percurso, troca de camadas, editor de mapas, filtros etc.).

Regras da apresentação

- Dia de apresentação é dia de avaliação: ausência, atraso excessivo ou saída prolongada da sala resultam em desconto. Não há segunda chamada.
- Perguntas individuais: qualquer integrante pode ser questionado sobre qualquer parte do projeto, e a resposta é avaliada individualmente.
- **As professoras podem escolher, na hora, quem apresenta. A nota desse integrante passa a ser a nota do grupo.** Por isso, todos precisam dominar todo o trabalho.
- Integrante que não está contribuindo deve ser informado às professoras por e-mail durante o desenvolvimento. Não será aceita a justificativa "fulano não ajudou" no dia da apresentação.

Uso de IA generativa

O uso de IA generativa é **permitido somente** nas situações abaixo e **sempre declarado** na Declaração de Uso de IA (modelo fornecido pela professora).

- **Permitido (com registro):** tirar dúvidas conceituais; entender erros e depurar código escrito pelo grupo; sugerir casos de teste e mapas extras, conferidos pelo grupo; apoio com pandas e bibliotecas de visualização; revisão de ortografia e clareza de textos do grupo.
- **Proibido:** gerar a implementação de BFS, DFS, Dijkstra, Bellman-Ford ou da construção do grafo de estados; gerar análises, discussões, conclusões, notas analíticas ou a prova do m05; **pedir que a IA interprete ou "ajude a analisar" gráficos, escolha o tipo de gráfico ou os elementos de destaque, ou monte a história da análise explanatória**; pedir que a IA resolva os mapas; entregar trecho que ninguém do grupo saiba explicar.
- **Obrigatório:** marcar no código os trechos feitos com apoio de IA (`# [IA] <ferramenta> – <finalidade>`) e entregar a declaração assinada por todos (assinatura manuscrita ou digital, ex.: gov.br), **mesmo que o grupo não tenha usado IA**. O registro de cada uso pode ser substituído pelo compartilhamento dos links das conversas completas com as professoras.

Uso não declarado, declaração ausente, ou código/análise que o grupo não sabe explicar na arguição: nota ZERO no projeto.

Observações e dicas importantes

- **O mapa não é o grafo.** Antes de programar, desenhem à mão no papel o `G_est` do m02 e confiram com o código.
- **Comecem pelos mapas pequenos** (m01, m02) e só depois passem para o m03 e o m08.
- **Ordem dos vizinhos:** sempre N, L, S, O. A classificação de arcos da DFS depende disso.
- **Explosão de estados:** não gerem as 2^k combinações de antemão; explorem a partir do início.
- **Grafo dirigido:** não espelhem arcos.
- **Documentem tudo no PDF:** modelagem do estado, prova do m05, correção do m06, limitações.
- **Sem libs de algoritmo:** toda a lógica de BFS/DFS/Dijkstra/BF deve ser **implementação própria** (ok usar `heapq` em Dijkstra).

AVISO FINAL:

Este projeto é interdisciplinar e será utilizado também na disciplina de Análise e Visualização de Dados (AVD).

Os elementos da Parte 2 relacionados à exploração, visualização e comunicação de dados serão avaliados pela docente de AVD, conforme os critérios acima.